

ECE 457A — Adaptive and Cooperative Algorithms

Population-Based Algorithms Real-Valued and Permutation Genetic Algorithms

University of Waterloo

June 19, 2026

Real-Valued Genetic Algorithms

Direct evolutionary search in continuous spaces

Insight

Representation and operators are adapted to $\mathbb{R}^n$.

Why Move Beyond Binary Encoding?

Many optimization problems are *inherently continuous*:

$$\mathbf{x} \in \mathbb{R}^n$$

Typical examples include:

- ▶ Control and tuning parameters
- ▶ Filter and model coefficients
- ▶ Engineering design variables

When applied to such problems, **binary encoding** introduces:

- ▶ **Artificial discretization** of a continuous space
- ▶ **Long chromosomes** to achieve acceptable precision
- ▶ **Distorted neighborhood structure**, weakening locality

Note

For continuous problems, binary encoding adds complexity without adding insight.

Precision vs. Chromosome Length

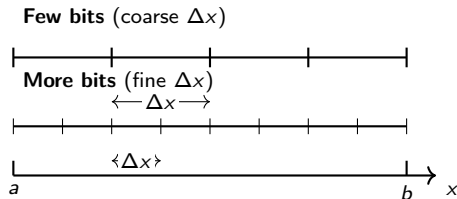
Consider a variable $x \in [a, b]$ encoded using n bits:

$$\Delta x = \frac{b - a}{2^n - 1}$$

Implications:

- ▶ Higher precision requires rapidly increasing n
- ▶ Each added bit **doubles** the search space
- ▶ In d dimensions, chromosome length is nd
- ▶ Search space grows as 2^{nd}

$\uparrow n \Rightarrow \downarrow \Delta x$
but $|S|$ doubles each bit



Note

Binary precision becomes costly quickly, especially in high dimensions.

Beyond Binary Encodings

So far, we assumed chromosomes are represented as **bit strings**.

However, many optimization problems are **naturally**:

- ▶ **Continuous** rather than discrete
- ▶ **Real-valued**, not symbolic
- ▶ Defined over **smooth parameter spaces**

This raises a key design question:

Why force a binary abstraction when the problem is not binary?

Alternative encodings allow:

- ▶ Direct representation of decision variables
- ▶ Variation operators that respect problem geometry
- ▶ More efficient exploration of large, continuous spaces

Note

Binary GA is foundational—but representation should match the problem.

How we encode solutions matters

Representation determines:

- ▶ What neighborhoods look like
- ▶ How variation operators behave
- ▶ How efficiently the search explores the space

Note

There is no universal encoding—only appropriate ones.

Common Genetic Representations

Binary Encoding

101101001011

- ▶ Discrete abstraction
- ▶ Bit-flip mutation
- ▶ Historically foundational

Real-Valued Encoding

[1.27 -0.53 4.02]

- ▶ Genotype = phenotype
- ▶ Smooth variation
- ▶ Natural for $\mathbb{R}^n$

Permutation Encoding

$\langle 4, 1, 5, 2, 3 \rangle$

- ▶ Ordering problems
- ▶ Constraint-preserving operators
- ▶ Combinatorial structure

Note

Different problem structures demand different encodings.

Real-Valued Chromosome Representation

A solution is represented **directly** as a real-valued vector:

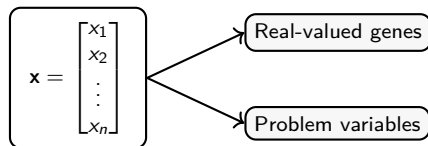
$$\mathbf{x} = (x_1, x_2, \dots, x_n), \quad x_i \in \mathbb{R}$$

Each gene corresponds to a problem variable:

- ▶ Controller and tuning gains
- ▶ Engineering design parameters
- ▶ Continuous decision variables

Note

Real-valued GA works directly in the problem's native space.



No encoding / decoding step

Key property:

- ▶ **Genotype = Phenotype**
- ▶ No binary mapping or discretization
- ▶ Natural neighborhood structure

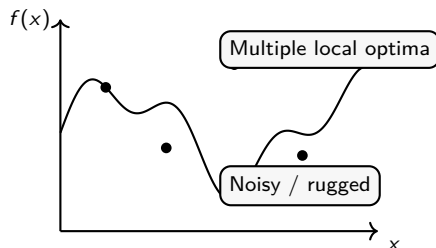
When Real-Valued GA Is Preferred

Real-valued GA is particularly effective when:

- ▶ The objective function is **continuous**
- ▶ Gradient information is **unavailable**, unreliable, or expensive
- ▶ The search landscape is **multimodal**, noisy, or rugged
- ▶ Global exploration is more important than local optimality

Note

Real-valued GA excels when smooth structure exists but gradients cannot be trusted.



Typical application domains:

- ▶ Control and parameter tuning
- ▶ Engineering and system design
- ▶ Hyperparameter optimization in ML

Constraints in Real-Valued Genetic Algorithms

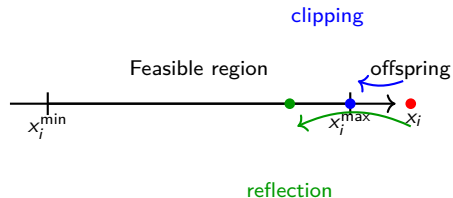
Most real-valued optimization problems impose **box constraints**:

$$x_i^{\min} \leq x_i \leq x_i^{\max}$$

After mutation or crossover, variables may violate bounds.

Common constraint-handling strategies:

- ▶ **Repair (clipping):** force values back to nearest bound
- ▶ **Reflection:** mirror excess back into the feasible region
- ▶ **Rejection & resampling:** discard and redraw offspring



Design trade-off:

- ▶ Simple handling vs. preserving search dynamics

Note

Constraint handling is part of the operator design.

Why Bit-Level Crossover Fails in Real-Valued Spaces

Bit-level crossover assumes a **symbolic** representation:

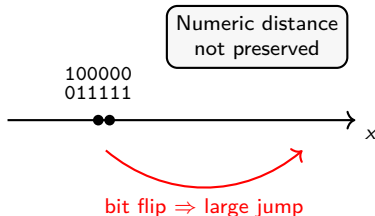
- ▶ Gene positions have no inherent geometry
- ▶ Neighborhoods are defined syntactically

When applied to real-valued problems:

- ▶ Small bit changes may cause **large numeric jumps**
- ▶ Similar chromosomes may encode **distant solutions**
- ▶ The geometry of the search space is ignored

Note

Variation must respect the geometry of $\mathbb{R}^n$.



Consequence:

- ▶ Disruptive variation
- ▶ Poor locality and slow convergence

Recombination in Continuous Search Spaces

- ▶ Operate directly on **real-valued chromosomes**
- ▶ Generate offspring via **arithmetic and geometric combinations**
- ▶ Support **smoother exploration** and **finer exploitation**

Key idea: crossover as interpolation, not bit swapping

Arithmetic Crossover

Given two parent solutions:

$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}$$

An offspring is generated as a convex combination:

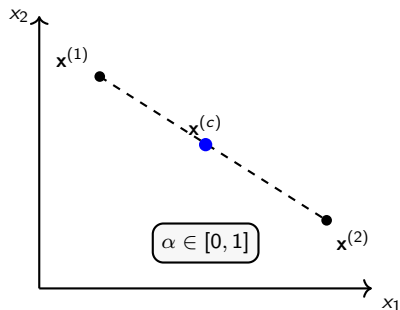
$$\mathbf{x}^{(c)} = \alpha \mathbf{x}^{(1)} + (1 - \alpha) \mathbf{x}^{(2)}, \quad \alpha \in [0, 1]$$

Key properties:

- ▶ Geometry-aware recombination
- ▶ Preserves feasibility for box constraints
- ▶ Interpolates between parent solutions

Note

Arithmetic crossover exploits the geometry of $\mathbb{R}^n$.



Arithmetic Crossover: Numeric Example

Given two parent solutions:

$$\mathbf{x}^{(1)} = (2.0, 4.0), \quad \mathbf{x}^{(2)} = (6.0, 1.0)$$

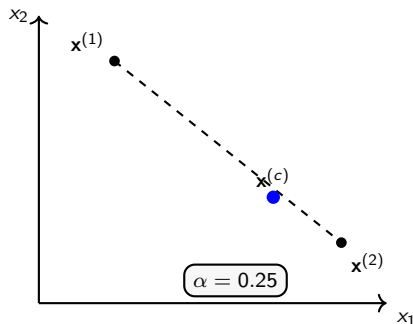
Let $\alpha = 0.25$.

Offspring computation:

$$\begin{aligned}\mathbf{x}^{(c)} &= \alpha \mathbf{x}^{(1)} + (1 - \alpha) \mathbf{x}^{(2)} \\ &= 0.25(2, 4) + 0.75(6, 1) \\ &= (5.0, 1.75)\end{aligned}$$

Note

Arithmetic crossover performs controlled interpolation.



Observation:

- ▶ Offspring is closer to $\mathbf{x}^{(2)}$
- ▶ Distance controlled by α

BLX- α Crossover: Numerical Example

Given one gene x_i from two parents:

$$x_i^{(1)} = 4.0, \quad x_i^{(2)} = 10.0, \quad \alpha = 0.5$$

Step 1: compute parent distance

$$d = |x_i^{(1)} - x_i^{(2)}| = |4.0 - 10.0| = 6$$

Step 2: BLX- α sampling interval

$$L = \min(x_i^{(1)}, x_i^{(2)}) - \alpha d = 4.0 - 0.5 \times 6 = 1.0$$

$$U = \max(x_i^{(1)}, x_i^{(2)}) + \alpha d = 10.0 + 0.5 \times 6 = 13.0$$

Step 3: sample offspring gene

$$x_i^{(c)} \sim \mathcal{U}[1.0, 13.0]$$

Example draw:

$$x_i^{(c)} = 11.2$$

Note

The offspring may lie *outside* the parent interval, controlled by α .

BLX- α Crossover: 4-Gene Example (Setup)

Parent chromosomes:

$$\mathbf{x}^{(1)} = (4.0, 2.0, 8.0, 5.0), \quad \mathbf{x}^{(2)} = (10.0, 6.0, 3.0, 9.0)$$

Crossover parameter:

$$\alpha = 0.5$$

BLX- α is applied **independently to each gene** x_i .

Gene i	$x_i^{(1)}$	$x_i^{(2)}$	Sampling Interval
1	4.0	10.0	[1.0, 13.0]
2	2.0	6.0	[0.0, 8.0]
3	8.0	3.0	[0.5, 10.5]
4	5.0	9.0	[3.0, 11.0]

Note

Each gene defines its own BLX- α sampling range.

Simulated Binary Crossover (SBX)

Simulated Binary Crossover (SBX) emulates the effect of **binary crossover** in real-valued spaces.

Key idea:

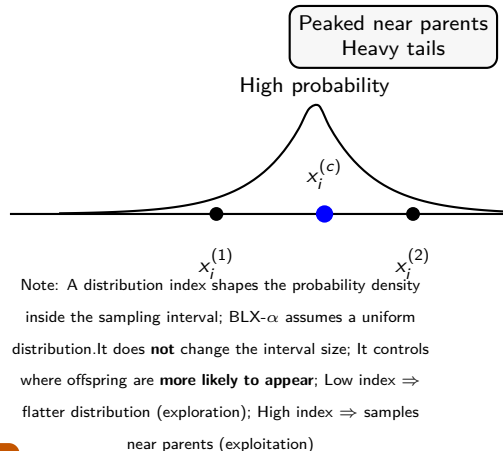
- ▶ High probability of **small perturbations**
- ▶ Low probability of **large jumps**
- ▶ Preserves locality while allowing exploration

Control parameter:

- ▶ Distribution index η_c
- ▶ Larger $\eta_c \Rightarrow$ offspring closer to parents
- ▶ Smaller $\eta_c \Rightarrow$ more exploratory behavior

Note

SBX reproduces binary-GA locality without bit strings.



BLX- α uses uniform sampling, so no distribution index is defined.

SBX Example: One Decision Variable

Example setup:

$$x_i^{(1)} = 2.0, \quad x_i^{(2)} = 6.0, \quad \eta_c = 2.$$

For one random number $u \in [0, 1]$, SBX computes

$$\beta_q = \begin{cases} (2u)^{\frac{1}{\eta_c+1}}, & u \leq 0.5, \\ \left(\frac{1}{2(1-u)}\right)^{\frac{1}{\eta_c+1}}, & u > 0.5. \end{cases}$$

Assume

$$u = 0.25.$$

Then

$$\beta_q = (2 \times 0.25)^{\frac{1}{3}} = (0.5)^{\frac{1}{3}} \approx 0.79.$$

Offspring generation:

$$c_i^{(1)} = \frac{1}{2} \left[(1 + \beta_q)x_i^{(1)} + (1 - \beta_q)x_i^{(2)} \right]$$

$$c_i^{(2)} = \frac{1}{2} \left[(1 - \beta_q)x_i^{(1)} + (1 + \beta_q)x_i^{(2)} \right]$$

Substituting $\beta_q \approx 0.79$:

$$c_i^{(1)} = \frac{1}{2} [1.79(2.0) + 0.21(6.0)] \approx 2.42$$

$$c_i^{(2)} = \frac{1}{2} [0.21(2.0) + 1.79(6.0)] \approx 5.58$$

Crossover Operators: Summary

Operator	Exploration	Exploitation	Complexity
Arithmetic	Low	High	Low
BLX- α	Medium	Medium	Medium
SBX	Adjustable	Adjustable	Medium

Note

Crossover choice should reflect problem structure and search stage.

Controlled Perturbations in Continuous Search Spaces

small, stochastic changes guide continuous adaptation

Role of Mutation in Real-Valued GA (Clarified)

Mutation acts on a single parent at a time.

Given a parent $\mathbf{x}$, mutation samples:

$$\mathbf{x}' = \mathbf{x} + \epsilon$$

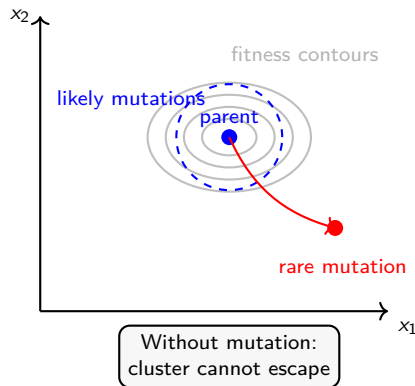
where ϵ is a random perturbation.

Key effects:

- ▶ High probability of **small local steps**
- ▶ Low probability of **larger exploratory jumps**
- ▶ Prevents population collapse into one basin

Note

Mutation is controlled randomness around a parent.



Gaussian Mutation

Gaussian mutation is the standard mutation operator for real-valued chromosomes.

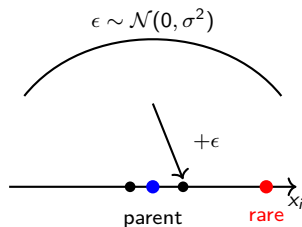
For each gene:

$$x'_i = x_i + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

Interpretation:

- ▶ Centered at the parent value x_i
- ▶ Most mutations are **small**
- ▶ Occasional larger steps are possible

Control knob: σ sets the mutation strength (step size).



Note

Gaussian noise produces smooth, local exploration around a parent.

Gaussian Mutation: Numeric Example

Parent solution:

$$\mathbf{x} = (3.0, -1.0)$$

Mutation model:

$$x'_i = x_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2), \quad \sigma = 0.2$$

One sampled noise vector:

$$\epsilon = (0.12, -0.05)$$

Offspring computation (gene by gene):

$$x'_1 = 3.0 + 0.12 = 3.12$$

$$x'_2 = -1.0 - 0.05 = -1.05$$

Resulting offspring:

$$\mathbf{x}' = (3.12, -1.05)$$

Note

Gaussian mutation introduces small, random, independent perturbations to each gene.

Adaptive Gaussian Mutation

Mutation strength may change over generations, while the mutation distribution remains Gaussian.

At generation g :

$$x'_i = x_i + \epsilon_i(g), \quad \epsilon_i(g) \sim \mathcal{N}(0, \sigma(g)^2)$$

Design intent:

- ▶ Large $\sigma(g)$ early $\rightarrow$ strong exploration
- ▶ Small $\sigma(g)$ later $\rightarrow$ fine exploitation

Typical schedule:

$$\sigma(g) \downarrow \quad \text{as} \quad g \uparrow$$

Note

The mutation distribution stays Gaussian; only its scale adapts over time.

Boundary Handling After Mutation

Real-valued mutation (e.g., Gaussian) may produce **infeasible values**:

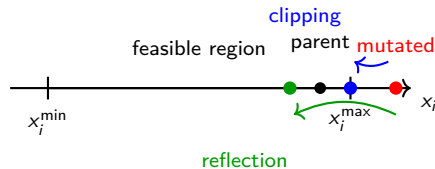
$$x_i < x_i^{\min} \quad \text{or} \quad x_i > x_i^{\max}$$

This is common when:

- ▶ Mutation strength σ is large (early generations)
- ▶ The parent is close to a boundary

Common boundary-handling strategies:

- ▶ **Clipping:** set $x_i = x_i^{\min}$ or $x_i^{\max}$
- ▶ **Reflection:** mirror the excess back into the range
- ▶ **Resampling:** redraw the mutated gene until feasible



Design note:

- ▶ Boundary handling interacts with mutation strength
- ▶ Poor handling can bias the search toward boundaries

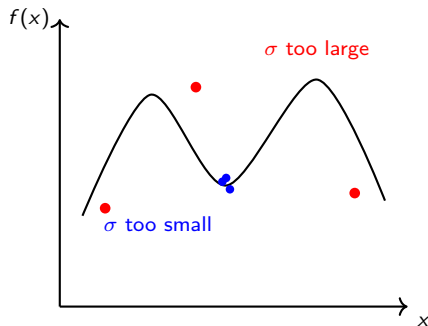
Note

Mutation generates candidates;
boundary handling restores feasibility.

Mutation Strength vs. Convergence

Effect of mutation strength σ :

- ▶ σ **too large**:
 - ▶ Random-walk behavior
 - ▶ Poor convergence
- ▶ σ **too small**:
 - ▶ Premature convergence
 - ▶ Loss of diversity



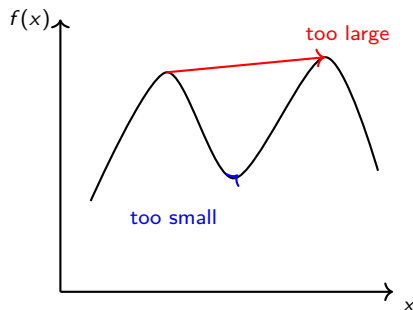
Note

Mutation strength must be carefully calibrated.

Common Pitfalls in Real-Valued Mutation

Frequent mistakes in mutation design:

- ▶ **Fixed mutation scale for all problems**
 - ▶ Step size mismatched to problem scale
- ▶ **Ignoring variable sensitivity**
 - ▶ Some variables require finer perturbations than others
- ▶ **Forgetting boundary repair**
 - ▶ Leads to infeasible or biased search



Note

Mutation design must reflect problem structure and constraints.

Example Problem

Minimize the function:

$$f(x, y) = (x - 2)^2 + (y + 1)^2$$

Properties:

- ▶ Continuous
- ▶ Convex
- ▶ Global minimum at $(2, -1)$

Note: This example illustrates GA mechanics, not algorithm choice.

Initialization

Initial population (size = 4):

Individual	x	y
1	0.5	-0.2
2	3.8	-2.0
3	1.5	1.0
4	4.2	-0.5

$$f(x, y) = (x - 2)^2 + (y + 1)^2$$

Fitness Evaluation

Evaluate each individual:

ID	x	y	$f(x, y)$
1	0.5	-0.2	2.89
2	3.8	-2.0	4.24
3	1.5	1.0	4.25
4	4.2	-0.5	5.09

$$f(x, y) = (x - 2)^2 + (y + 1)^2$$

Note

Lower fitness values are preferred (minimization).

Selection

ID	x	y	$f(x, y)$
1	0.5	-0.2	2.89
2	3.8	-2.0	4.24
3	1.5	1.0	4.25
4	4.2	-0.5	5.09

Assume tournament selection (size = 2).

Selected parents:

- ▶ Individual 1
- ▶ Individual 2

Note

Better solutions are favored but not guaranteed.

Crossover Step

Apply arithmetic crossover with $\alpha = 0.4$:

$$\mathbf{x}^{(c)} = 0.4(0.5, -0.2) + 0.6(3.8, -2.0)$$

$$\mathbf{x}^{(c)} = (2.48, -1.28)$$

Offspring lies on the line segment between parents.

Mutation Step

Apply Gaussian mutation with $\sigma = 0.1$:

$$\Delta = (-0.03, 0.05) \quad (\text{one Gaussian draw})$$

$$\mathbf{x}' = (2.45, -1.23)$$

Note

Small perturbations refine solutions locally.

New Population Member

New offspring:

$$\mathbf{x}' = (2.45, -1.23)$$

Fitness:

$$f(2.45, -1.23) = 0.25$$

Note

GA operators rapidly move solutions toward the optimum.

One-Generation Summary

In one generation:

- ▶ Poor solutions are discarded
- ▶ Better offspring are introduced
- ▶ Population shifts toward the optimum

Note

Evolution is incremental but directional.

Why Real-Valued GA Works Well Here

- ▶ Search space is continuous
- ▶ Operators preserve geometry
- ▶ Mutation enables fine local refinement

Note

Real-valued GA aligns naturally with the problem structure.

Permutation Genetic Algorithms

Evolutionary search over ordered, discrete structures

Insight

Validity of solutions must be preserved by specialized operators.

Permutation-Based Representations

Some problems are not about values—but about **order**.

A chromosome is a permutation:

[3, 1, 4, 2, 5]

Each element appears:

- ▶ Exactly once
- ▶ In a specific order

Note

Validity is a hard constraint.

Where Permutation GAs Arise

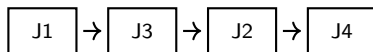
Common applications:

- ▶ Traveling Salesman Problem (TSP)
- ▶ Scheduling and sequencing
- ▶ Routing and ordering tasks

Objective depends on:

- ▶ **Relative order** of elements
- ▶ **Adjacency relationships** between elements

4-Machine Scheduling Order



Only the order matters
Adjacency affects cost

Note

The order itself encodes the solution.

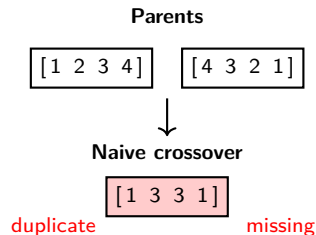
Why Standard Crossover Fails

Binary or real-valued crossover, when applied naively to permutations, may produce:

- ▶ **Duplicate elements**
- ▶ **Missing elements**
- ▶ **Invalid permutations**

These operators:

- ▶ Do not preserve ordering constraints
- ▶ Treat positions independently



Note

Permutation problems require constraint-preserving crossover.

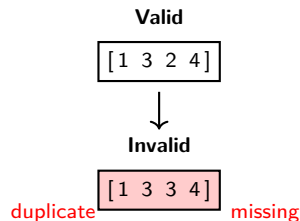
Key Constraint in Permutation GA

A valid permutation must satisfy:

$$\forall i \neq j: x_i \neq x_j$$

Implications for operators:

- ▶ **Preserve uniqueness** of all elements
- ▶ **Preserve relative ordering** where possible



Violating this constraint leads to:

- ▶ Duplicate elements
- ▶ Missing elements
- ▶ Infeasible solutions

Insight

All variation operators must be permutation-aware.

Permutation Crossover Operators

Crossover operators that preserve ordering and feasibility

Insight

Crossover must generate valid permutations at all times.

Partially Matched Crossover (PMX)

In permutation GA (TSP, scheduling, routing), solutions must remain **valid permutations**.

Problem with naive crossover:

- ▶ duplicates appear (same city/job repeated)
- ▶ some elements go missing
- ▶ offspring becomes infeasible

PMX idea:

- ▶ **copy a segment** to preserve useful positional structure
- ▶ **repair conflicts** systematically so every element appears exactly once

Note

PMX preserves positions while guaranteeing a valid permutation.

PMX: Procedure (Clean and Precise)

Given parents P_1, P_2 (permutations of the same set):

1. Choose two cut points $[c_1, c_2]$.
2. Create child C by copying the segment $P_1[c_1 : c_2]$ into $C[c_1 : c_2]$.
3. For each position $k \in [c_1, c_2]$, form a paired correspondence

$$P_1[k] \leftrightarrow P_2[k]$$

4. Fill positions outside the segment using P_2 , but if a value conflicts with the copied segment, resolve it by following the correspondence chain until a non-conflicting value is found.

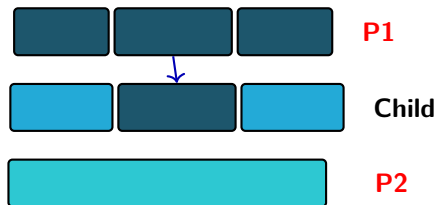
Note

Copy segment $\rightarrow$ build correspondences $\rightarrow$ fill + repair duplicates.

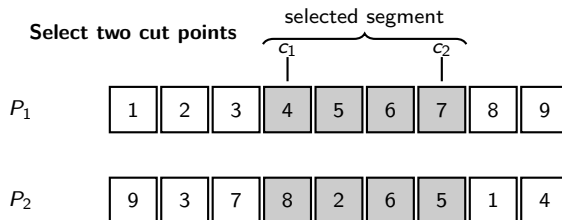
Permutations GAs — TSP Example

PMX: Informal procedure for parents P1 and P2:

1. Choose random segment in child and copy it from P1,
2. Starting from the first crossover point look for elements in that segment of P2 that have not been copied,
3. For each of these i , look in the offspring to see what element j has been copied in its place from P1,
4. Place i into the position occupied j in P2, since we know that we will not be putting j there (as is already in offspring).
5. If the place occupied by j in P2 has already been filled in the offspring k , put i in the position occupied by k in P2,
6. Having dealt with the elements from the crossover segment, the rest of the offspring can be filled from P2. Second child is created analogously.



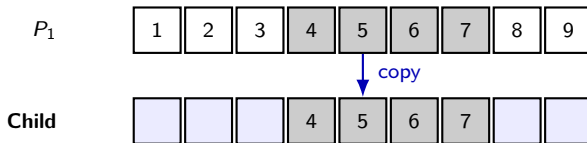
PMX Step 1: Choose Two Cut Points



Same cut points in both parents define the block of positions.

PMX Step 2: Copy Segment into the Child

Copy the selected segment from P_1



Child now has a fixed block (4–7). Remaining slots are empty.

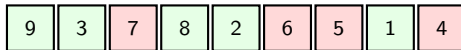
PMX Step 3: Identify Conflicts Using P_2

Scan P_2 : which values can fill the empty slots?

Child



P_2



Green = safe (not in copied block $\{4, 5, 6, 7\}$)

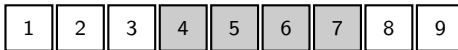
Red = conflict (already in the copied block)

Next: use PMX correspondences to *redirect* conflicts.

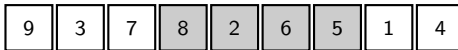
Select two cut points

selected segment

P_1



P_2



PMX Step 4: Conflict Repair by Correspondence Chaining

Context

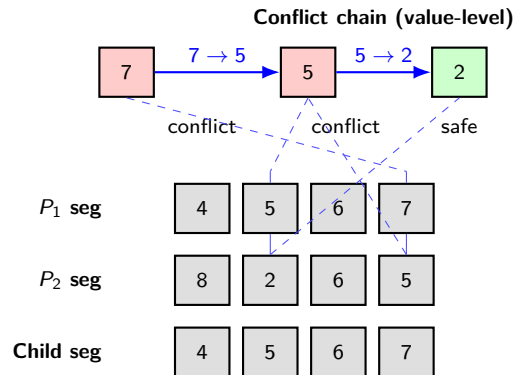
- ▶ Copied segment from P_1 is fixed in the child:
 $\{4, 5, 6, 7\}$
- ▶ These values **cannot appear elsewhere** in the child.

Attempted placement

- ▶ From P_2 , we attempt to place value 7.
- ▶ Since 7 is already in the copied segment, this is a **conflict**.

PMX repair rule

- ▶ Look up the correspondence induced by the copied segment:
 $7 \leftrightarrow 5, \quad 5 \leftrightarrow 2$
- ▶ Replace the conflicting value by following the correspondence.
- ▶ Continue until a value **not in the segment** is found.

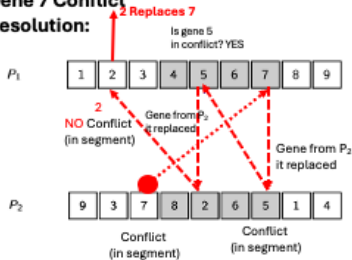


Each arrow follows the correspondence in the copied segment: find the column with the current value in P_1 , then read its paired value in P_2 ;

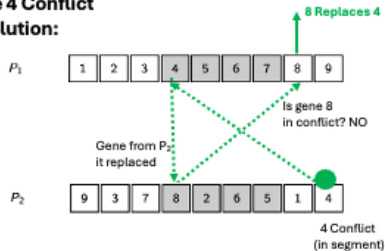
Step 4: Visual Summary of Conflict Resolution



Gene 7 Conflict Resolution:



Gene 4 Conflict Resolution:



Worked Example: Permutation GA on a 7-City TSP

Goal: minimize tour length (return to start)

Cities: $\{A, B, C, D, E, F, G\}$

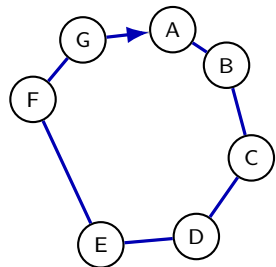
Chromosome (permutation):

$$\pi = [\pi_1, \pi_2, \dots, \pi_7]$$

$$L(\pi) = \sum_{k=1}^6 d(\pi_k, \pi_{k+1}) + d(\pi_7, \pi_1)$$

We will demonstrate one generation:

- ▶ Evaluate fitness (tour length)
- ▶ Tournament selection
- ▶ PMX crossover (step-by-step)
- ▶ Swap mutation
- ▶ Replacement with elitism



tour = permutation + return

Note

Permutation GA requires permutation-aware crossover (PMX).

Distance Data (Example Instance)

Symmetric distance matrix
(units arbitrary)

	A	B	C	D	E	F	G
A	0	4	3	7	3	6	8
B	4	0	4	6	7	3	5
C	3	4	0	5	6	7	4
D	7	6	5	0	2	4	3
E	3	7	6	2	0	5	6
F	6	3	7	4	5	0	4
G	8	5	4	3	6	4	0

Encoding for PMX

$A = 1, B = 2, C = 3, D = 4, E = 5, F = 6, G = 7$

Fitness

$$L(\pi)$$

Objective

minimize $L(\pi)$

Note

TSP fitness is a sum along the permutation edges.

Initialization: Population (Size 4) + Fitness

Population $P^{(0)}$ (minimize $L(\pi)$)

- ▶ P_1 : [1, 2, 3, 4, 5, 6, 7] $L = 32$
- ▶ P_2 : [1, 3, 2, 5, 4, 7, 6] $L = 29$
- ▶ P_3 : [2, 6, 4, 5, 1, 3, 7] $L = 24$ ← best
- ▶ P_4 : [3, 1, 5, 2, 7, 4, 6] $L = 32$

Best tour P_3



$L = 24$

Permutation
(no repeats)

Note

Lower tour length = better fitness (minimization).

Selection: Tournament (Size 2) — Visual

Assume two tournaments:

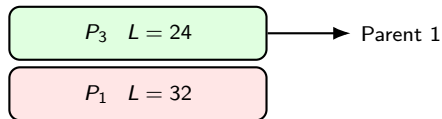
- ▶ Tournament A: P_3 vs $P_1 \Rightarrow$ winner P_3
- ▶ Tournament B: P_2 vs $P_4 \Rightarrow$ winner P_2

So parents are:

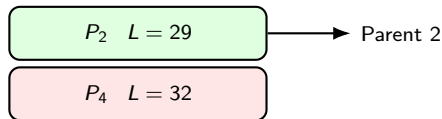
Parent 1 = P_3 = [2, 6, 4, 5, 1, 3, 7]

Parent 2 = P_2 = [1, 3, 2, 5, 4, 7, 6]

Tournament A



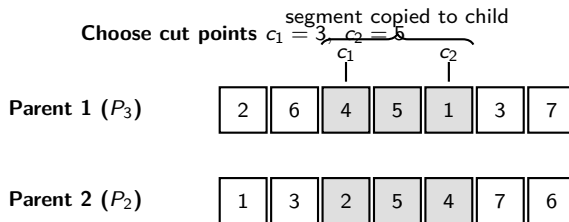
Tournament B



Note

Selection biases toward short tours, but keeps diversity.

PMX Step 1: Choose Cut Points and Mark Segments



Note

PMX copies a contiguous segment (positions) from Parent 1.

PMX Step 2: Copy Segment to Child + Build the Mapping

Copy the segment from P_1 (positions 3..5) into the child:

$$C = [-, -, \boxed{4, 5, 1}, -, -, -, -]$$

Build the PMX mapping from the swapped blocks (value-to-value):

$$P_1[3..5] = [4, 5, 1], \quad P_2[3..5] = [2, 5, 4]$$

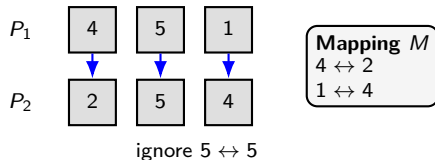
$$M : \quad 4 \leftrightarrow 2, \quad 1 \leftrightarrow 4$$

($5 \leftrightarrow 5$ is trivial and can be ignored.)

Note

The copied segment fixes positions; the segment pairs define the repair mapping.

Swapped blocks (pos 3..5)



How it is used (repair intuition):

- ▶ When we try to place a value from P_2 outside the segment,
- ▶ if it **duplicates** something already in the child,
- ▶ we **follow the mapping M** (possibly chaining) until we get a value not in the segment.

PMX Step 3: Fill Remaining Positions from Parent 2 (with Repair)

Child after copy:

$$C = [-, -, 4, 5, 1, -, -]$$

Copied set $S = \{4, 5, 1\}$.

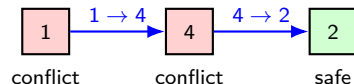
Now fill positions 1, 2, 6, 7 from Parent 2: $[1, 3, -, -, -, 7, 6]$

Key conflict (example at pos 1):

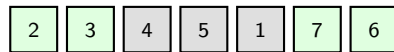
- ▶ try 1 $\Rightarrow$ conflict ($1 \in S$)
- ▶ map $1 \rightarrow 4$ (still conflict)
- ▶ map $4 \rightarrow 2$ (safe) $\Rightarrow$ place 2

Then place remaining safe values in order:

$$C = [2, 3, 4, 5, 1, 7, 6]$$



Child



Mutation: Swap Two Positions (Optional Diversification)

Assume swap mutation triggers once.

Before mutation:

$$C = [2, 3, 4, 5, 1, 7, 6]$$

Swap positions 2 and 6:

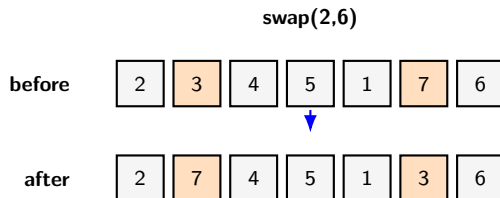
$$C' = [2, 7, 4, 5, 1, 3, 6]$$

Interpretation (letters):

$$[B, G, D, E, A, C, F]$$

Note

Swap mutation preserves permutation feasibility and injects diversity.



Offspring Fitness + Replacement (Elitism)

Evaluate mutated offspring C' :

$$C' = [2, 7, 4, 5, 1, 3, 6] \Rightarrow L(C') = 26$$

Elitist replacement (keep best 1):

Keep P_3 ($L = 24$)

Then fill remaining slots with best available:

$$\{P_3(24), C'(26), P_2(29), P_1(32)\}$$

Drop worst: $P_4(32)$.

Note

Elitism prevents losing the best tour while still exploring via offspring.

Next population

elite: P_3 (24)

offspring: C' (26)

P_2 (29)

P_1 (32)

drop: P_4 (32)

PMX Example 1: Simple Mapping

Parents

$$P_1 = [1\ 2\ 3\ 4\ 5\ 6\ 7\ 8]$$

$$P_2 = [5\ 7\ 2\ 1\ 8\ 4\ 6\ 3]$$

Choose crossover points after positions 3 and 6.

$$P_1 = [1\ 2\ 3\ |\ 4\ 5\ 6\ |\ 7\ 8]$$

$$P_2 = [5\ 7\ 2\ |\ 1\ 8\ 4\ |\ 6\ 3]$$

Copy the segment from P_1 :

$$C = [-\ -\ -\ |\ 4\ 5\ 6\ |\ -\ -]$$

PMX mapping

$$4 \leftrightarrow 1, \quad 5 \leftrightarrow 8, \quad 6 \leftrightarrow 4$$

Fill remaining positions from P_2 :

$$[5, 7, 2, 4, 5, 6, 6, 3]$$

Duplicates must be repaired:

$$5 \rightarrow 8, \quad 6 \rightarrow 4 \rightarrow 1$$

$$C = [8\ 7\ 2\ 4\ 5\ 6\ 1\ 3]$$

PMX Example 2: Mapping Chain

Parents

$$P_1 = [1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9]$$

$$P_2 = [9\ 3\ 7\ 8\ 2\ 6\ 5\ 1\ 4]$$

Choose crossover points after positions 3 and 7.

$$P_1 = [1\ 2\ 3\ |\ 4\ 5\ 6\ 7\ |\ 8\ 9]$$

$$P_2 = [9\ 3\ 7\ |\ 8\ 2\ 6\ 5\ |\ 1\ 4]$$

Copy the segment from P_1 :

$$C = [-\ -\ -\ |\ 4\ 5\ 6\ 7\ |\ -\ -]$$

PMX Mapping

$$4 \leftrightarrow 8$$

$$5 \leftrightarrow 2$$

$$7 \leftrightarrow 5$$

Remaining values from P_2

$$[9, 3, 7, 1, 4]$$

Conflict repair

$$7 \rightarrow 5 \rightarrow 2$$

$$4 \rightarrow 8$$

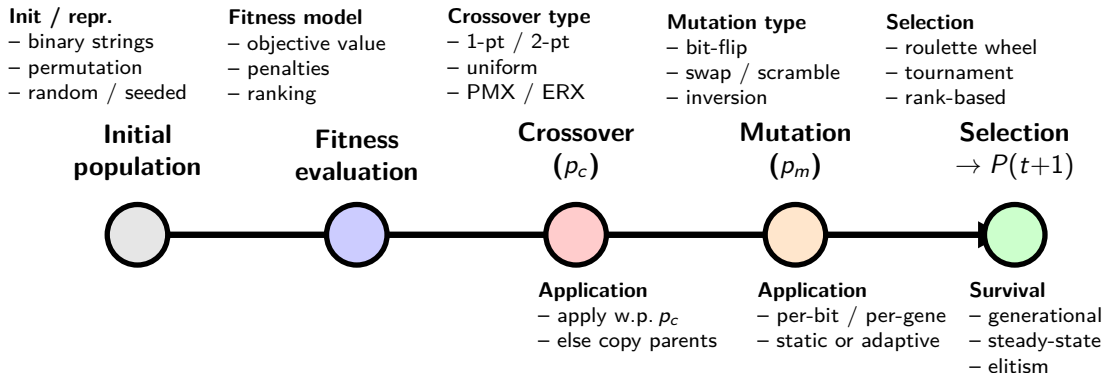
Final offspring

$$C = [9\ 3\ 2\ 4\ 5\ 6\ 7\ 1\ 8]$$

Key Observation

When a duplicate occurs, PMX follows the mapping chain until an unused value is found.

Genetic Algorithm: Order + Strategy Choices



Story: evaluate $\rightarrow$ crossover w.p. $p_c \rightarrow$ mutation w.p. $p_m \rightarrow$ stochastic selection $\rightarrow$ repeat.